

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЁТ
по ИПР №2
по дисциплине
«Интернет-технологии и веб-программирование»
Вариант 19

Выполнил

ст. гр. 220604
М.В. Пашковец

Проверил

Н.В. Хаджинова

Минск 2025

1 ВВЕДЕНИЕ И ЦЕЛЬ РАБОТЫ

В современную эпоху цифровой трансформации, характеризующуюся переходом к распределенным и микросервисным архитектурам, интерфейсы прикладного программирования (*Application Programming Interfaces, API*) выступают в качестве фундаментальной основы для взаимодействия между различными программными компонентами. Они обеспечивают эффективный и стандартизированный обмен данными, выступая связующим звеном между клиентскими приложениями, такими как одностраничные приложения (*SPA*) и мобильные клиенты, и серверными сервисами. Роль *API* не ограничивается внутренним взаимодействием в рамках одной системы, но также включает предоставление публичных интерфейсов, позволяющих сторонним приложениям и разработчикам получать программный доступ к данным и функциональности платформы. Эта способность *API* обеспечивать интеграцию и расширяемость систем делает их краеугольным камнем современных программных экосистем, где взаимодействие между сервисами становится важным.

Среди множества архитектурных стилей для построения веб-сервисов подход *Representational State Transfer (REST)* стал де-факто доминирующим стандартом. *REST*, базирующийся на принципах протокола *HTTP*, предлагает набор архитектурных стандартов и ограничений, таких как единообразие интерфейса, отсутствие состояния, кэшируемость и клиент-серверное разделение ответственности. Эти принципы способствуют созданию масштабируемых, гибких и платформенно-независимых решений. *RESTful API* позволяют достичь высокой производительности и надежности за счет стандартизации использования *HTTP*-методов для выполнения операций с ресурсами и обеспечения четкого разделения ответственности между клиентом, который управляет пользовательским интерфейсом, и сервером, который отвечает за хранение данных и бизнес-логику.

Выбор чистого *PHP* для реализации такого *API*, в отличие от использования полнофункциональных фреймворков, предоставляет глубокое понимание основ работы веб-сервисов. Данный подход, хотя и требует большего внимания к деталям архитектуры, таким как самостоятельная реализация шаблона «Единая точка входа» (*Front Controller*) и механизма маршрутизации, позволяет разработчику полностью контролировать процесс обработки *HTTP*-запросов и формирования ответов. Это обеспечивает строгое соблюдение принципов *REST* без наслоений абстракций, характерных для высокоуровневых фреймворков, и способствует созданию легковесного, производительного решения, архитектура которого прозрачна и полностью

управляема.

Целью работы является разработка и практическая реализация масштабируемого, безопасного и не сохраняющего состояние (*stateless*) *RESTful API* на языке *PHP* для управления сущностью «Вакансии» в реляционной базе данных. Разрабатываемое программное обеспечение должно предоставлять стандартизированный интерфейс для выполнения полного набора операций над данными, обеспечивая при этом высокий уровень контроля над процессами взаимодействия между клиентом и сервером.

Ключевым аспектом работы является обеспечение надежной безопасности системы. Это достигается за счет реализации обязательного механизма аутентификации на основе *API*-ключа, передаваемого в заголовке *X-API-Key*, и применения подготовленных запросов через *PDO* для полного устранения уязвимостей, связанных с *SQL*-инъекциями. Все операции обмена данными должны осуществляться исключительно в формате *JSON*, что соответствует современным стандартам построения веб-сервисов.

Архитектура *API* строится по принципу единой точки входа, что позволяет централизованно управлять маршрутизацией запросов и обработкой входящих данных. Важной задачей является проектирование и реализация детализированной системы обработки ошибок, которая возвращает семантически корректные *HTTP*-статусы для всех возможных сценариев, включая успешное выполнение операций, ошибки клиента и сбои на сервере.

2 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ АРХИТЕКТУРЫ

Архитектура *RESTful API* основана на фундаментальных ограничениях стиля *REST* (*Representational State Transfer*), которые обеспечивают структурированный подход к проектированию распределенных систем. Эти ограничения формируют основу для создания масштабируемых, надежных и поддерживаемых веб-сервисов, способных эффективно функционировать в гетерогенных средах.

Одним из ключевых принципов *REST* является разделение ответственности между клиентом и сервером. Данный принцип предполагает четкое разделение задач: клиентская часть отвечает за пользовательский интерфейс и представление данных, тогда как серверная часть сосредоточена исключительно на хранении, обработке данных и реализации бизнес-логики. Такое разделение позволяет клиентским и серверным компонентам развиваться независимо друг от друга, что значительно упрощает процессы масштабирования и обслуживания системы. Практической реализацией этого принципа в *PHP*-приложениях является использование архитектуры *Front*

Controller с выделенным слоем моделей для работы с данными, что обеспечивает строгое отделение бизнес-логики от механизмов обработки *HTTP*-запросов.

Важнейшим аспектом *REST*-архитектуры является принцип отсутствия состояния (*statelessness*). Согласно этому принципу, каждый запрос от клиента к серверу должен содержать всю необходимую информацию для его обработки, без зависимости от предыдущих запросов. Сервер не должен хранить какое-либо состояние сессии между запросами, что обеспечивает существенные преимущества в масштабируемости и надежности системы. Практическим следствием этого принципа является необходимость передачи аутентификационных данных (например, *API*-ключа в заголовке *X-API-Key*) с каждым запросом, что позволяет серверу независимо обрабатывать каждый запрос и эффективно распределять нагрузку между множеством серверных экземпляров.

Принцип единообразия интерфейса (*uniform interface*) представляет собой комплексное ограничение, реализуемое через несколько взаимосвязанных аспектов. Во-первых, ресурсы должны быть однозначно идентифицированы через *URI* (*Uniform Resource Identifiers*), что обеспечивает стандартизированный доступ к данным. Во-вторых, манипулирование ресурсами осуществляется через их представления - клиент работает с представлением ресурса (например, в формате *JSON*) для определения способов изменения состояния ресурса на сервере. В-третьих, сообщения должны быть самоописывающимися, содержащими всю необходимую метаданную для их корректной обработки.

Особое место в системе принципов *REST* занимает концепция *HATEOAS* (*Hypermedia as the Engine of Application State*), которая требует, чтобы клиент мог динамически обнаруживать все доступные действия и переходы через гипермедиа-ссылки, встроенные в тело ответа. Это позволяет серверу гибко управлять состоянием приложения и эволюционировать без нарушения работы существующих клиентов.

Архитектурный стиль *REST* фундаментально опирается на протокол *HTTP* не только как на транспортный механизм, но и как на неотъемлемую часть своей концептуальной модели. Это глубокое взаимодействие проявляется в использовании стандартных *HTTP*-методов для выполнения операций с ресурсами, где каждому методу присваивается строго определенное семантическое назначение, напрямую сопоставляемое с операциями *CRUD*. Такое соответствие обеспечивает единообразие интерфейса и предсказуемость поведения *API*.

Протокол *HTTP* предоставляет развитую систему кодов состояния, которые играют *crucial* роль в *RESTful API*, обеспечивая семантически насыщенную коммуникацию между клиентом и сервером. Правильное использование этих кодов позволяет клиентским приложениям точно интерпретировать результат выполнения запроса и соответствующим образом реагировать на различные сценарии.

Метод *GET* предназначен исключительно для безопасного чтения

данных и не должен изменять состояние ресурса на сервере. Его идемпотентность гарантирует, что множество идентичных запросов приведут к одному и тому же результату без побочных эффектов в системе. В контексте *CRUD* он реализует операцию *Read*, позволяя клиентам получать как коллекции ресурсов, так и отдельные их экземпляры через соответствующие *URI*.

Методы *PUT* и *PATCH* обеспечивают функциональность обновления существующих ресурсов, но с различной семантикой. *PUT* предполагает полную замену ресурса предоставленным представлением, являясь идемпотентной операцией – многократное выполнение одного и того же запроса *PUT* оставляет ресурс в одинаковом состоянии. В отличие от него, *PATCH* предназначен для частичного обновления и не является идемпотентным, так как последовательность нескольких частичных изменений может давать различный результат.

Коды состояния из класса 2xx сигнализируют об успешном выполнении запроса. Статус 200 *OK* указывает на стандартный успешный ответ для методов *GET*, *PUT* или *PATCH*, когда сервер возвращает запрошенные данные или подтверждает выполнение операции. Статус 201 *Created* имеет более специфическое назначение – он возвращается при успешном создании нового ресурса методом *POST* и должен сопровождаться заголовком *Location* с *URI* созданного ресурса, что позволяет клиенту немедленно получить к нему доступ (рисунок 1).

Для создания новых ресурсов используется метод *POST*, который не является ни безопасным, ни идемпотентным. Каждый успешный запрос *POST* создает новый ресурс, что приводит к изменению состояния сервера. Этот метод обычно применяется в операциях *Create*, когда клиент передает в теле запроса данные для формирования нового ресурса, при этом сервер самостоятельно назначает ему уникальный идентификатор и *URI*.

Метод *DELETE* обеспечивает удаление указанного ресурса и обладает свойством идемпотентности. Повторные запросы *DELETE* к одному и тому же ресурсу должны возвращать одинаковый результат – обычно код 404 *Not Found* после первого успешного удаления, что не считается ошибкой, а подтверждает отсутствие ресурса.

Коды класса 4xx обозначают клиентские ошибки, когда проблема заключается в некорректно сформированном запросе. Статус 400 *Bad Request* указывает на общую синтаксическую ошибку в запросе, такую как невалидный *JSON* или отсутствующие обязательные поля. Статус 401 *Unauthorized* возвращается когда аутентификация не удалась – например, при отсутствии, истечении срока действия или неверном значении *API*-ключа в заголовке *X-API-Key*. Статус 403 *Forbidden* применяется когда клиент аутентифицирован, но не имеет прав доступа к запрашиваемому ресурсу.

Статус *404 Not Found* – один из наиболее распространенных кодов, указывающий что запрашиваемый ресурс не существует по указанному *URI*, что особенно важно в операциях *GET*, *PUT*, *PATCH* и *DELETE* с несуществующими идентификаторами (рисунок 1).

HTTP Status Codes



Рисунок 1 – *HTTP* статус-коды

Коды класса *5xx* сигнализируют о серверных ошибках, когда проблема возникает на стороне сервера независимо от корректности клиентского запроса. Статус *500 Internal Server Error* представляет общую ошибку сервера когда он сталкивается с непредвиденной ситуацией, которая не позволяет выполнить запрос – например, сбой подключения к базе данных или исключения в бизнес-логике. Этот статус не должен использоваться для обработки клиентских ошибок или проблем с валидацией данных (рисунок 1).

JavaScript Object Notation утвердился в качестве стандарта де-факто для обмена данными в современных *RESTful API* благодаря оптимальному сочетанию простоты, читаемости и эффективности. Этот текстовый формат, основанный на подмножестве синтаксиса *JavaScript*, обеспечивает легковесную альтернативу более громоздким форматам вроде *XML*, что значительно снижает нагрузку на сеть и ускоряет обработку данных.

Структурная простота *JSON* проявляется в использовании пар «ключ-значение», организованных в объекты и массивы, что интуитивно понятно для разработчиков и легко обрабатывается средствами большинства языков программирования. В контексте *PHP* поддержка *JSON* реализована на уровне стандартных функций *json_encode()* и *json_decode()*, обеспечивающих эффективную сериализацию и десериализацию данных с минимальными накладными расходами.

3 ХОД РАБОТЫ

Процесс разработки разделен на три ключевых этапа: развертывание

среды, проектирование структуры проекта и реализация серверной логики обработки данных с акцентом на безопасность. Первым и фундаментальным этапом выступает настройка изолированной и воспроизводимой среды выполнения, что является важным условием для обеспечения консистентности разработки, тестирования и последующего развертывания приложения.

3.1 Установка и настройка *Docker Desktop*

В рамках данной работы принято решение об использовании технологии контейнеризации на базе платформы *Docker* для развертывания всей необходимой инфраструктуры. Данный подход позволяет полностью отказаться от традиционных методов локальной установки и настройки таких компонентов, как веб-сервер *Apache*, интерпретатор *PHP* и система управления базами данных *MySQL*, непосредственно на операционную систему разработчика. Подобные традиционные подходы, например, использование готовых пакетов вроде *XAMPP* или *WAMP*, часто сопряжены с проблемами конфликта версий программного обеспечения, сложностями конфигурирования, а также отсутствием гарантий идентичности среды на разных машинах, что существенно затрудняет командную работу и перенос проекта.

Технология контейнеризации предлагает принципиально иной уровень абстракции, инкапсулируя приложение со всеми его зависимостями в стандартизированную единицу – контейнер. Контейнер представляет собой изолированное пользовательское пространство внутри операционной системы, которое совместно использует ядро ОС хоста, но обладает собственной файловой системой, сетевыми интерфейсами и ограниченным доступом к ресурсам. Ключевыми преимуществами данного подхода являются полная изоляция сред, их легкая воспроизводимость и высочайшая степень портативности. Изоляция гарантирует, что процессы, выполняющиеся внутри контейнера, не влияют на процессы в хост-системе или в других контейнерах, что исключает конфликты версий и зависимостей. Воспроизводимость достигается за счет описания состава среды в виде кода – в специальных конфигурационных файлах, что позволяет любой машине с установленным *Docker* создать идентичную среду выполнения. Портативность означает, что образ контейнера, собранный на одной машине, может быть запущен без изменений на любой другой системе, поддерживающей *Docker*, будь то *Windows*, *macOS* или *Linux*.

Для управления многоконтейнерными приложениями, подобными

разрабатываемому в данной работе, где требуется одновременная работа веб-сервера и базы данных, используется инструмент *Docker Compose*, однако не запрещено использовать несколько *Dockerfile*, но для упрощения принят формат *compose*. *Docker Compose* – это утилита, которая позволяет определять и запускать сложные приложения, состоящие из нескольких контейнеров, с помощью единого декларативного файла конфигурации в формате *YAML*. В этом файле, по традиции именуемом *docker-compose.yml*, разработчик описывает все сервисы, которые составляют приложение, их базовые образы, зависимости между ними, переменные окружения, правила проброса портов и точки монтирования томов для хранения данных. Стоит отметить, что при необходимости настройки безопасной работы приложений может создаваться собственная сеть, внутри которой будут работать контейнеры. Использование *Docker Compose* кардинально упрощает координацию работы нескольких сервисов, так как вместо ручного запуска каждого контейнера отдельными командами, вся инфраструктура поднимается одной командой *docker compose up*. Это обеспечивает согласованное, предсказуемое и быстрое развертывание всей среды разработки или тестирования.

Помимо использования готовых образов из публичных реестров, таких как *Docker Hub*, часто возникает необходимость в их кастомизации под конкретные нужды проекта. Для этих целей используется *Dockerfile*. *Dockerfile* – это текстовый файл с набором инструкций, которые пошагово описывают процесс сборки нового *Docker*-образа. Каждая инструкция в *Dockerfile* создает новый слой в файловой системе образа. В контексте данной работы пользовательский *Dockerfile* создается на основе официального образа *php:8.2-apache*. Внутри этого файла выполняются критически важные действия, такие как установка дополнительных расширений *PHP* с помощью утилиты *docker-php-ext-install*. В частности, устанавливаются расширения *pdo*, *pdo_mysql* и *mysqli*, без которых становится невозможным безопасное взаимодействие *PHP*-скриптов с базой данных *MySQL*.

Процесс подготовки к запуску проекта требует минимальных предварительных шагов. Основным и единственным обязательным программным обеспечением, которое необходимо установить на компьютер разработчика, является *Docker Desktop*, его можно скачать на официальном сайте, там же можно найти множество уже готовых образов: <https://docs.docker.com/desktop/>. Отобразим это на рисунке 2.

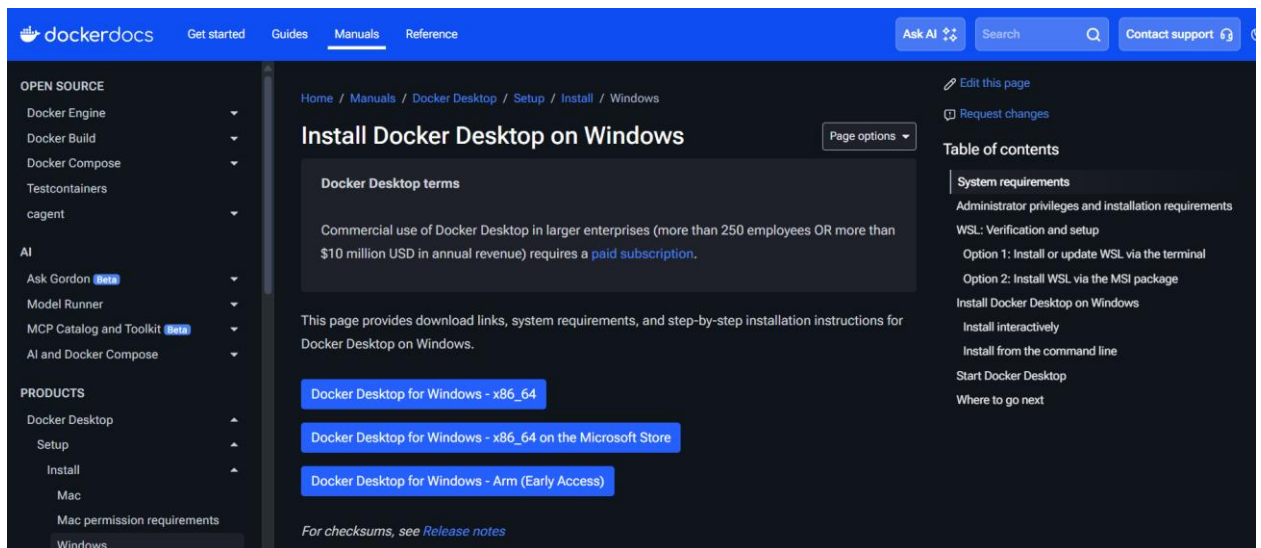


Рисунок 2 – Скачивание *Docker desktop*

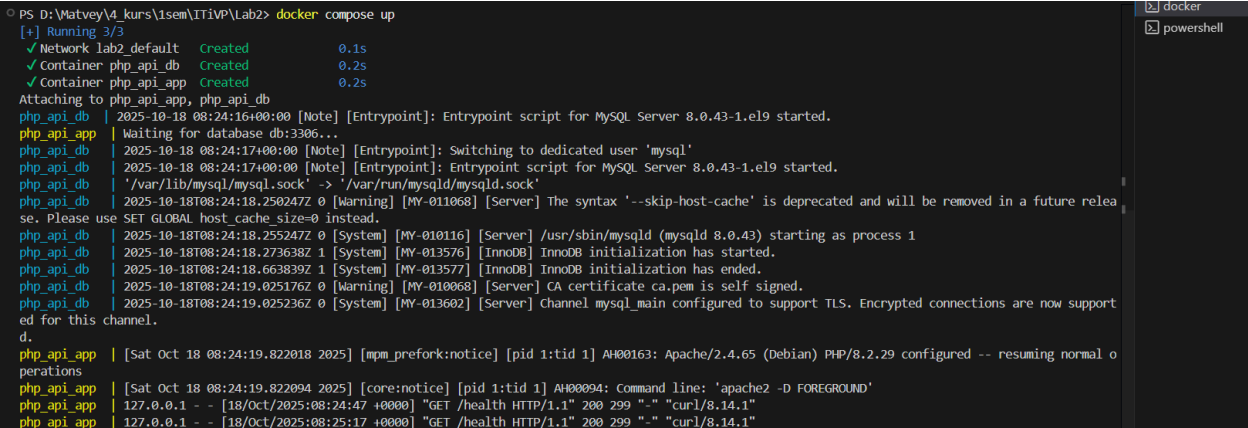
Docker Desktop представляет собой интегрированное решение, которое включает в себя как демон *Docker*, так и все необходимые сопутствующие утилиты, включая *Docker Compose*. После успешной установки и запуска *Docker Desktop* вся дальнейшая работа по управлению контейнерами и образами осуществляется через командную строку или графический интерфейс самого *Docker*. Наличие установленного *Docker* является единственным системным требованием; отпадает необходимость в самостоятельной установке и настройке *PHP*, веб-сервера *Apache* или СУБД *MySQL*, так как эти компоненты будут доставлены и запущены внутри своих изолированных контейнеров.

После выполнения команды *docker compose up* в терминале *PowerShell* из директории проекта (*PS D:\Matvey\4_kurs\1sem\ITiVP\Lab2>*) инициируется процесс развертывания контейнерной среды. В отличие от фонового режима с флагом *-d*, данный подход обеспечивает непосредственный вывод логов всех сервисов в текущую консоль, что позволяет в реальном времени отслеживать процесс запуска и диагностировать возможные проблемы.

Система последовательно создает сетевой мост *lab2_default* и два контейнера: *php_api_db* (база данных) и *php_api_app* (веб-приложение). В логах контейнера *php_api_db* детально отображается процесс инициализации СУБД *MySQL* 8.0.43: инициализация *InnoDB*, настройка *TLS*-соединений и другие этапы загрузки. Параллельно контейнер приложения *php_api_app* ожидает доступности базы данных на порту 3306, что демонстрирует реализованную в композе зависимость сервисов.

После успешной загрузки всех компонентов в логах появляется сообщение о готовности *Apache* 2.4.65 с *PHP* 8.2.29 к обработке запросов.

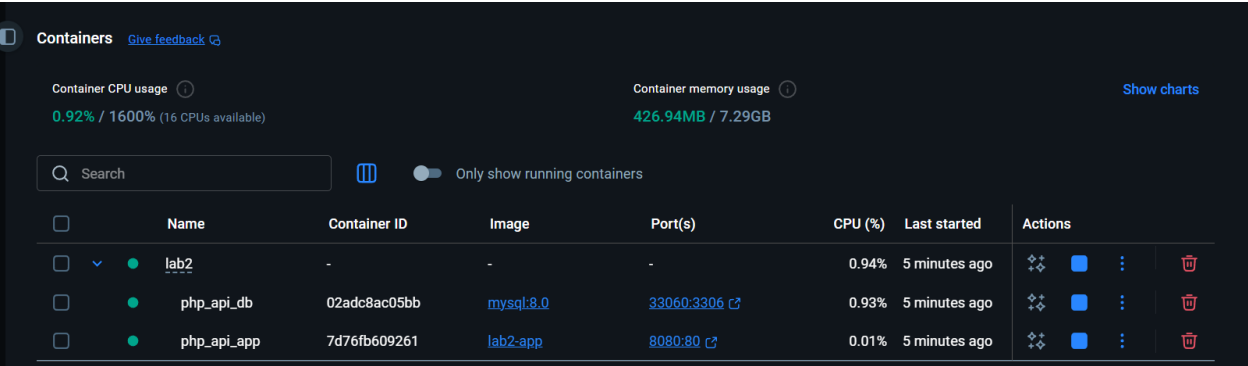
Фактическое подтверждение работоспособности системы демонстрирует запрос к эндпоинту `/health`, возвращающий статус 200, что свидетельствует о корректной работе веб-сервера и возможности обработки входящих *HTTP*-запросов. Отобразим вывод консоли на рисунке 3.



```
PS D:\Matvey\4_kurs\1sem\ITiVP\Lab2> docker compose up
[+] Running 3/3
 ✓ Network lab2_default Created 0.1s
 ✓ Container php_api_db Created 0.2s
 ✓ Container php_api_app Created 0.2s
Attaching to php_api_app, php_api_db
php_api_db | 2025-10-18 08:24:16+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.43-1.el9 started.
php_api_app | Waiting for database db:3306...
php_api_db | 2025-10-18 08:24:17+00:00 [Note] [Entrypoint]: Switching to dedicated user 'mysql'
php_api_db | 2025-10-18 08:24:17+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.43-1.el9 started.
php_api_db | '/var/lib/mysql/mysql.sock' -> '/var/run/mysqld/mysqld.sock'
php_api_db | 2025-10-18T08:24:18.258247Z 0 [Warning] [MY-011068] [Server] The syntax '--skip-host-cache' is deprecated and will be removed in a future release. Please use SET GLOBAL host_cache_size=0 instead.
php_api_db | 2025-10-18T08:24:18.255247Z 0 [System] [MY-010116] [Server] /usr/sbin/mysqld (mysqld 8.0.43) starting as process 1
php_api_db | 2025-10-18T08:24:18.273638Z 1 [System] [MY-013576] [InnoDB] InnoDB initialization has started.
php_api_db | 2025-10-18T08:24:18.663839Z 1 [System] [MY-013577] [InnoDB] InnoDB initialization has ended.
php_api_db | 2025-10-18T08:24:19.025176Z 0 [Warning] [MY-010068] [Server] CA certificate ca.pem is self signed.
php_api_db | 2025-10-18T08:24:19.025236Z 0 [System] [MY-013602] [Server] Channel mysql_main configured to support TLS. Encrypted connections are now supported for this channel.
php_api_app | [Sat Oct 18 08:24:19.822018 2025] [mpm_prefork:notice] [pid 1:tid 1] AH00163: Apache/2.4.65 (Debian) PHP/8.2.29 configured -- resuming normal operations
php_api_app | [Sat Oct 18 08:24:19.822094 2025] [core:notice] [pid 1:tid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
php_api_app | 127.0.0.1 - - [18/Oct/2025:08:24:47 +0000] "GET /health HTTP/1.1" 200 299 "-" "curl/8.14.1"
php_api_app | 127.0.0.1 - - [18/Oct/2025:08:25:17 +0000] "GET /health HTTP/1.1" 200 299 "-" "curl/8.14.1"
```

Рисунок 3 – Запуск Контейнера из консоли

После чего в приложении *Docker* появится отображение запущенного контейнера, что показано на рисунке 4.



Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
lab2	-	-	-	0.94%	5 minutes ago	[Stop] [Restart] [Refresh] [Delete]
php_api_db	02adc8ac05bb	mysql:8.0	33060:3306	0.93%	5 minutes ago	[Stop] [Restart] [Refresh] [Delete]
php_api_app	7d76fb609261	lab2-app	8080:80	0.01%	5 minutes ago	[Stop] [Restart] [Refresh] [Delete]

Рисунок 4 – Информация о запущенном контейнере

На рисунке выше представлена информация о запущенном контейнере: что он в себя включает и подробная информация о его компонентах, например, название, состояние, образ лежащий в основании, проброс портов, оказываемая на процессор нагрузка, затраты оперативной памяти, а также время запуска.

3.2 Проектирование базы данных

Проектирование и разработка структуры базы данных представляют

собой критически важный начальный этап создания любого веб-приложения, работающего с персистентными данными. Качественно спроектированная схема данных является залогом целостности хранимой информации, высокой производительности приложения и простоты его дальнейшего сопровождения и масштабирования. На данном этапе осуществляется перевод требований предметной области в формализованную структуру, понятную для системы управления базами данных.

3.2.1 Схема таблицы аутентификации (*api_keys*). Таблица *api_keys* предназначена для управления ключами доступа к *API* и реализует механизм аутентификации. Особое внимание уделено безопасности хранения конфиденциальной информации. Поле *api_key_hash* хранит односторонний хеш *API*-ключа, сгенерированный с помощью алгоритма *Bcrypt* через функцию *password_hash()*. Данный подход гарантирует, что даже в случае компрометации базы данных исходные ключи не могут быть восстановлены. Для оптимизации процесса верификации добавлено поле *key_prefix*, содержащее первые символы ключа, что позволяет выполнять быстрый предварительный поиск перед проведением криптографически ресурсоемкой операции проверки через *password_verify()*.

Поле *user_id* обеспечивает гибкость системы, позволяя ассоциировать ключи доступа с конкретными пользователями или приложениями. Флаг *is_active* обеспечивает контроль над действительностью ключей, позволяя оперативно отзывать доступ без физического удаления записей из базы данных. Поле *created_at* фиксирует момент создания ключа, что важно для аудита и мониторинга использования *API*.

3.2.2 Схема таблицы ресурсов (*vacancies*). Таблица *vacancies* спроектирована для эффективного хранения информации о вакансиях и поддерживает полный цикл *CRUD*-операций. Обязательное поле *title* обеспечивает базовую идентификацию вакансии, в то время как поля *description* и *requirements*, использующие тип данных *TEXT*, позволяют хранить объемные текстовые данные без искусственных ограничений по длине. Поле *salary_range* предоставляет возможность указыватьвилку заработной платы в формате, удобном для представления.

Особого внимания заслуживает система временных меток, реализованная в таблице. Поле *created_at* автоматически устанавливается в момент создания записи с помощью значения *DEFAULT CURRENT_TIMESTAMP*, что обеспечивает точную фиксацию времени добавления вакансии в систему. Более сложную логику реализует поле *updated_at*, которое изначально имеет значение *NULL*, указывая на то, что запись еще не подвергалась изменениям после создания.

Механизм обновления *updated_at* активируется только после успешного выполнения транзакции, направленной на изменение данных вакансии. Использование конструкции *ON UPDATE CURRENT_TIMESTAMP* гарантирует, что временная метка автоматически обновляется до текущего времени сервера при любом изменении записи. Эта двухфазная система временных меток предоставляет ценную метаинформацию: разница между *created_at* и *updated_at* позволяет отслеживать историю изменений вакансии, а наличие значения в *updated_at* свидетельствует о том, что запись хотя бы один раз редактировалась после первоначального создания.

3.2.3 Технические особенности реализации. Обе таблицы используют движок *InnoDB*, что обеспечивает поддержку транзакций, внешних ключей и механизма *row-level locking*. Кодировка *utf8mb4* с правилом сортировки *utf8mb4_unicode_ci* гарантирует корректное хранение и обработку текстовых данных на различных языках, включая специальные символы и эмодзи. Автоинкрементные первичные ключи обеспечивают уникальную идентификацию записей и оптимизируют операции соединения таблиц.

3.3 Архитектура приложения и реализация

3.3.1 Реализация паттерна *Front Controller* в *index.php*. Архитектура приложения построена на принципе единой точки входа (*Front Controller*), где файл *index.php* выполняет роль центрального диспетчера, обрабатывающего все входящие *HTTP*-запросы. Это достигается за счет настройки веб-сервера, который перенаправляет запросы на этот скрипт через механизм маршрутизации.

Инициализация приложения начинается с загрузки необходимых зависимостей и конфигураций. Строгая типизация *declare(strict_types=1)* обеспечивает повышенный уровень контроля типов данных на этапе компиляции. Автозагрузка классов организована через явное подключение файлов с использованием абсолютных путей, что гарантирует надежную работу системы независимо от текущей рабочей директории.

Важным аспектом архитектуры является разделение ответственности между *API* и пользовательским интерфейсом. При обращении к корневому пути (/) или */ui* система переключается в режим отдачи статического контента, отправляя *HTML*-страницу с соответствующим *Content-Type*. Это демонстрирует гибкость подхода, позволяющего совмещать *REST API* с традиционным веб-интерфейсом в рамках одного приложения.

Для *API*-эндпоинтов реализована двухуровневая система проверок. Первичная проверка гарантирует, что запрос начинается с */api*, возвращая

ошибку 404 для некорректных путей. Затем устанавливается соединение с базой данных через статический метод *Database::getConnection()*, который инкапсулирует логику подключения и обработки ошибок соединения.

3.3.2 Система аутентификации и модульная структура. Механизм аутентификации реализован в классе *Auth* и выполняет проверку API-ключа для всех запросов к защищенным эндпоинтам. Ключ извлекается из заголовков запроса с учетом регистронезависимости, что обеспечивает совместимость с различными клиентами. Особенностью реализации является массовая проверка всех активных ключей из базы данных с использованием функции *password_verify()*, что позволяет избежать временных атак на основе сравнения времени ответа.

Модульная структура проекта организована по функциональному признаку. В директории */config* размещены классы конфигурации: *Database* отвечает за управление подключениями к базе данных, используя переменные окружения для гибкой настройки параметров подключения, а *Auth* инкапсулирует логику аутентификации. Класс *VacancyModel* в директории */models* представляет слой работы с данными, обеспечивая абстракцию для выполнения *CRUD*-операций с вакансиями.

3.3.3 Маршрутизация и обработка запросов. Система маршрутизации анализирует *HTTP*-метод и структуру *URI* для определения запрашиваемой операции. Для работы с *JSON*-данными реализована замыкающая функция *\$readJson*, которая обеспечивает безопасное чтение и парсинг тела запроса с валидацией формата данных.

Обработка *GET*-запросов демонстрирует два сценария: получение списка всех вакансий при обращении к */api/vacancies* и получение конкретной вакансии по идентификатору. Система выполняет строгую проверку валидности идентификатора и существования записи, возвращая семантически корректные *HTTP*-статусы (400 для невалидного *ID*, 404 для отсутствующей записи).

Операции создания и обновления вакансий включают комплексную валидацию входных данных. Обязательное поле *title* проверяется на наличие и непустое значение, в то время как остальные поля могут быть опциональными. При успешном создании вакансии возвращается статус 201 *Created* с данными созданной записи, включая автоматически сгенерированный идентификатор.

Для операций обновления поддерживаются как *PUT*, так и *PATCH* методы, что обеспечивает гибкость взаимодействия с API. Перед выполнением обновления система проверяет существование целевой записи, предотвращая модификацию несуществующих данных. Операция удаления также включает предварительную проверку существования вакансии, что гарантирует

идемпотентность операции *DELETE*.

Все операции с базой данных используют подготовленные выражения (*prepared statements*), что обеспечивает защиту от *SQL*-инъекций и повышает производительность при повторяющихся запросах. Модель данных возвращает результаты в формате ассоциативных массивов, что упрощает их последующую сериализацию в *JSON*.

3.4 Описание конечных точек (*Endpoints*) *API*

Архитектура *RESTful API* реализует принцип единообразия интерфейса через систему конечных точек, основанных на ресурсах и *HTTP*-методах. Все эндпоинты используют *URI* на основе существительных во множественном числе (*/api/vacancies*), что соответствует общепринятым соглашениям *REST* и обеспечивает интуитивно понятную структуру *API* для разработчиков.

3.4.1 Сводная карта конечных точек. Система предоставляет полный набор операций *CRUD* для управления ресурсом «Вакансии», где каждый *HTTP*-метод выполняет строго определенную семантическую функцию согласно стандартам протокола *HTTP*.

Операция чтения списка вакансий реализована через метод *GET* к эндпоинту */api/vacancies*. Данная конечная точка возвращает полный перечень вакансий, отсортированный в порядке убывания идентификаторов, что обеспечивает получение самых свежих записей в начале списка. Ответ сервера имеет статус 200 *OK* и содержит массив объектов вакансий в формате *JSON*, включая все поля записи: идентификатор, заголовок, описание, требования, диапазон заработной платы и временные метки.

Операция чтения конкретной вакансии выполняется через метод *GET* к эндпоинту */api/vacancies/{id}*, где идентификатор вакансии передается как часть *URI*. Система выполняет валидацию параметра *ID*, проверяя его корректность и существование соответствующей записи в базе данных. При успешном выполнении запроса возвращается статус 200 *OK* с полным объектом вакансии. В случае невалидного идентификатора система возвращает ошибку 400 *Bad Request*, а для несуществующей записи - 404 *Not Found*.

Операция создания новой вакансии реализована через метод *POST* к эндпоинту */api/vacancies*. Запрос должен содержать тело в формате *JSON* с обязательным полем «*title*» и опциональными полями «*description*», «*requirements*» и «*salary_range*». Система выполняет строгую проверку наличия и корректности обязательных полей, возвращая ошибку 400 *Bad Request* при нарушении требований валидации. Успешное создание вакансии подтверждается статусом 201 *Created*, а в теле ответа возвращается созданный

объект с присвоенным идентификатором.

Операция обновления вакансии поддерживается через методы *PUT* и *PATCH* к эндпоинту */api/vacancies/{id}*. Оба метода требуют передачи идентификатора в *URI* и полного *JSON*-тела с данными для обновления. Система проверяет существование целевой записи и валидность передаваемых данных, с обязательной проверкой поля «*title*». После успешного обновления возвращается статус 200 *OK* с актуальным состоянием вакансии, включая автоматическое обновление временной метки *updated_at* в базе данных.

Операция удаления вакансии выполняется через метод *DELETE* к эндпоинту */api/vacancies/{id}*. Как и в операциях чтения и обновления, система проверяет валидность идентификатора и существование записи перед выполнением удаления. После успешного удаления возвращается статус 200 *OK* с подтверждающим сообщением, хотя в строгом соответствии со стандартами *REST* предпочтительным статусом был бы 204 *No Content* без тела ответа.

Все конечные точки защищены механизмом аутентификации через *API*-ключ, который должен передаваться в заголовке *X-API-Key*. В случае отсутствия или невалидности ключа система возвращает статус 401 *Unauthorized*.

3.5 Валидация данных

Система валидации данных реализует многоуровневый подход к обеспечению целостности и корректности обрабатываемой информации. На первом уровне выполняется синтаксическая проверка *JSON*-структуры запроса. При получении данных система анализирует корректность *JSON*-формата через функцию *json_decode()* с последующей проверкой ошибок парсинга с использованием *json_last_error()*. В случае обнаружения синтаксических ошибок в *JSON* немедленно возвращается статус 400 *Bad Request* с описанием конкретной проблемы, что позволяет клиенту оперативно исправить формат запроса.

Семантическая валидация обязательных полей представляет собой второй критически важный уровень защиты. Для операций создания и обновления вакансий система проводит строгую проверку наличия и корректности поля *title*. Реализована комплексная проверка, включающая не только существование поля в передаваемых данных, но и его тип (строка) и фактическое наличие содержания после удаления пробельных символов. Такой подход исключает возможность создания вакансий с пустыми названиями, сохраняя целостность бизнес-логики приложения.

Третий уровень валидации обеспечивает корректность типов данных для всех передаваемых параметров. Система явно преобразует опциональные поля к ожидаемым типам, используя приведение типов (*string*) для текстовых полей *description*, *requirements* и *salary_range*. Это гарантирует, что даже при передаче числовых или других значений они будут корректно обработаны и сохранены в соответствующем строковом формате, предотвращая потенциальные ошибки выполнения запросов к базе данных.

Особое внимание уделяется валидации идентификаторов в *URI*. При операциях с конкретными ресурсами система выполняет строгую проверку параметра *ID*, обеспечивая его соответствие ожидаемому целочисленному формату. Перед выполнением операций обновления или удаления дополнительно проверяется существование записи с указанным идентификатором, что предотвращает выполнение операций над несуществующими данными и обеспечивает семантическую корректность ответов.

Система обеспечивает *graceful degradation* при возникновении внутренних ошибок сервера. В случае непредвиденных исключений на уровне базы данных или внутренней логики приложения клиенту возвращается статус *500 Internal Server Error* без раскрытия деталей реализации

3.6 Ответ на контрольный вопрос: Как вывести все ошибки валидации списком, а не по одной?

Аутентификация и авторизация – это фундаментальные понятия в области информационной безопасности, которые часто используются вместе, но решают разные задачи. Аутентификация отвечает на вопрос «Кто вы?» и представляет процесс проверки подлинности пользователя или системы. Например, ввод логина и пароля, использование биометрических данных или проверка цифрового сертификата – все это методы аутентификации. Её основная цель – удостовериться, что субъект действительно является тем, за кого себя выдает.

В отличие от этого, авторизация решает вопрос «Что вам разрешено делать?» после того, как личность была подтверждена. Этот процесс определяет уровень доступа пользователя к ресурсам системы: какие файлы он может просматривать, какие операции выполнять или к каким разделам приложения получать доступ. Например, разные пользователи в одной системе могут иметь различные права: один только просматривать данные, а другой – их изменять или удалять.

Таким образом, аутентификация предшествует авторизации и

устанавливает личность, в то время как авторизация устанавливает границы допустимых действий для этой личности. Простой аналогией может служить доступ в здание: аутентификация – это предъявление пропуска охране (подтверждение личности), а авторизация – определение, в какие именно помещения владелец пропуска может войти.

4 ДЕМОНСТРАЦИЯ РАБОТЫ ПРИЛОЖЕНИЯ

Продemonстрируем работу написанного *API*.

На рисунке 5 представлено начальное окно веб-интерфейса приложения, доступное по корневому адресу. Данный интерфейс предоставляет базовый пользовательский интерфейс для взаимодействия с системой. Для доступа к защищенным операциям с вакансиями пользователь должен ввести корректный *API*-ключ в соответствующее поле. После успешной аутентификации интерфейс предоставляет возможности для просмотра, создания, редактирования и удаления записей о вакансиях.

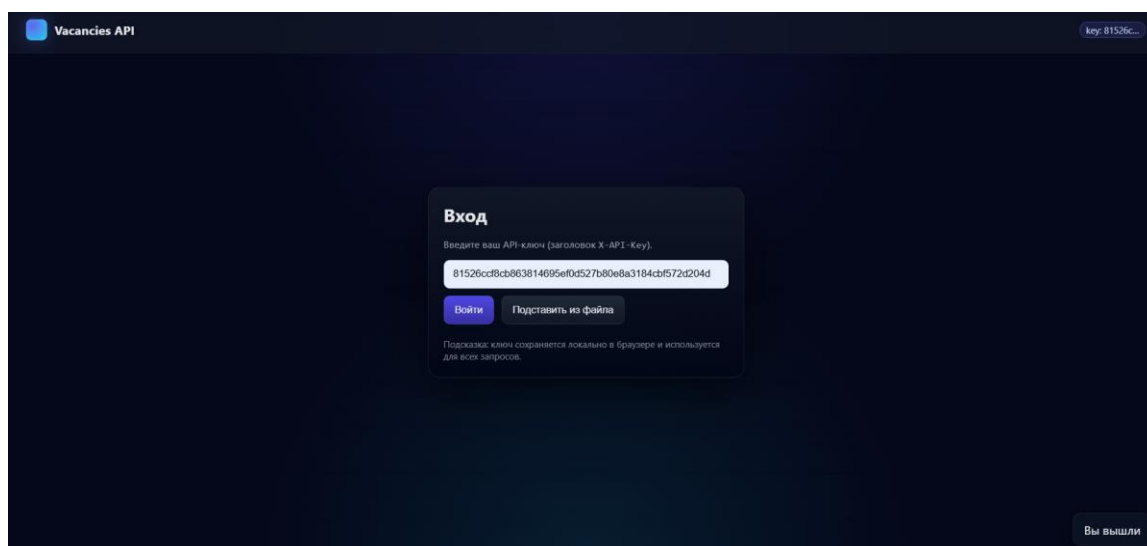


Рисунок 5 – Начальный экран веб-интерфейса

На рисунке 6 демонстрируется результат успешного выполнения запроса на получение списка всех вакансий через метод *GET* к эндпоинту */api/vacancies*. В ответе от сервера отображен массив *JSON*-объектов, каждый из которых содержит полную информацию о вакансии: уникальный идентификатор, название, описание, требования, вилку заработной платы, а также временные метки создания и последнего обновления. Данный запрос не требует передачи дополнительных параметров, но обязательно включает валидный *API*-ключ в заголовке.

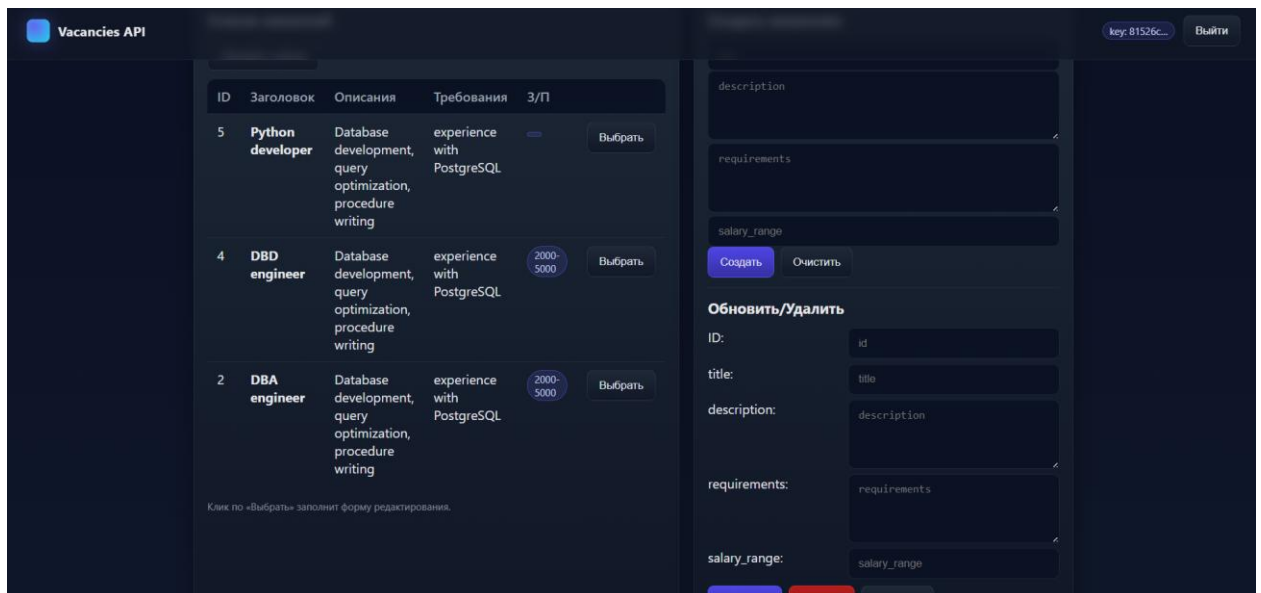


Рисунок 6 – Просмотр списка всех вакансий (*GET /api/vacancies*)

Рисунок 7 иллюстрирует процесс создания новой вакансии с использованием метода *POST* к эндпоинту */api/vacancies*. В теле запроса передается *JSON*-объект с обязательным полем «*title*» и опциональными полями «*description*», «*requirements*» и «*salary_range*». В ответе сервер возвращает статус 201 *Created* и полное представление созданного ресурса, включая автоматически присвоенный уникальный идентификатор (*id*), что подтверждает успешное выполнение операции.

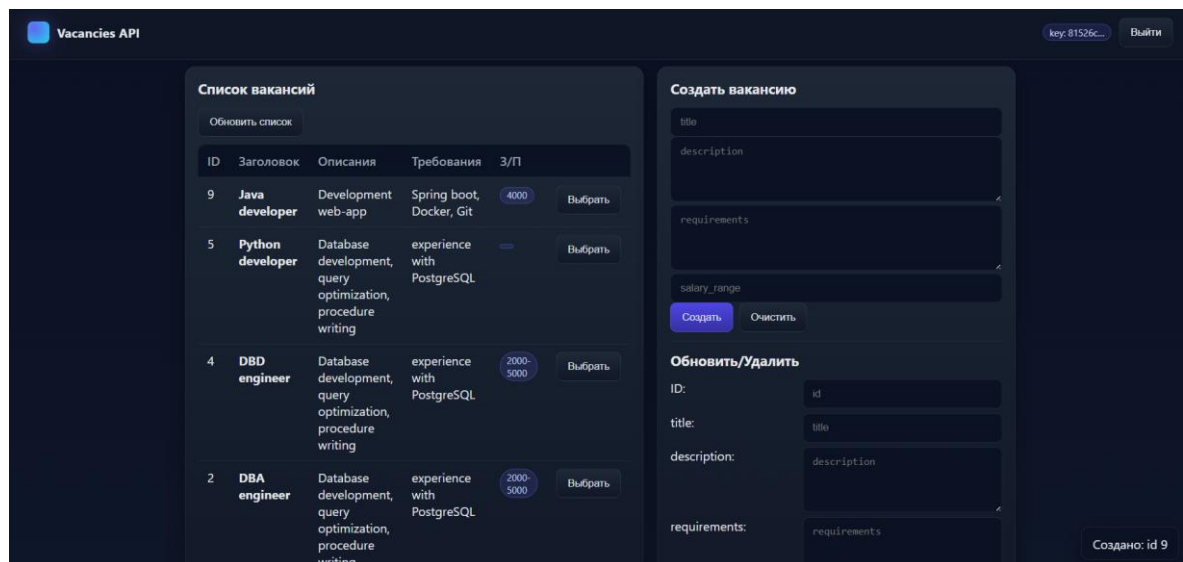


Рисунок 7 – Серверная валидация

Рисунок 8 демонстрирует операцию полного обновления данных вакансии с помощью метода *PUT*. Запрос направляется к эндпоинту */api/vacancies/{id}* и содержит в теле полное представление ресурса, включая

все обязательные и опциональные поля. В ответе сервер возвращает статус 200 OK и актуальное состояние обновленной вакансии, при этом в базе данных автоматически обновляется временная метка *updated_at*.

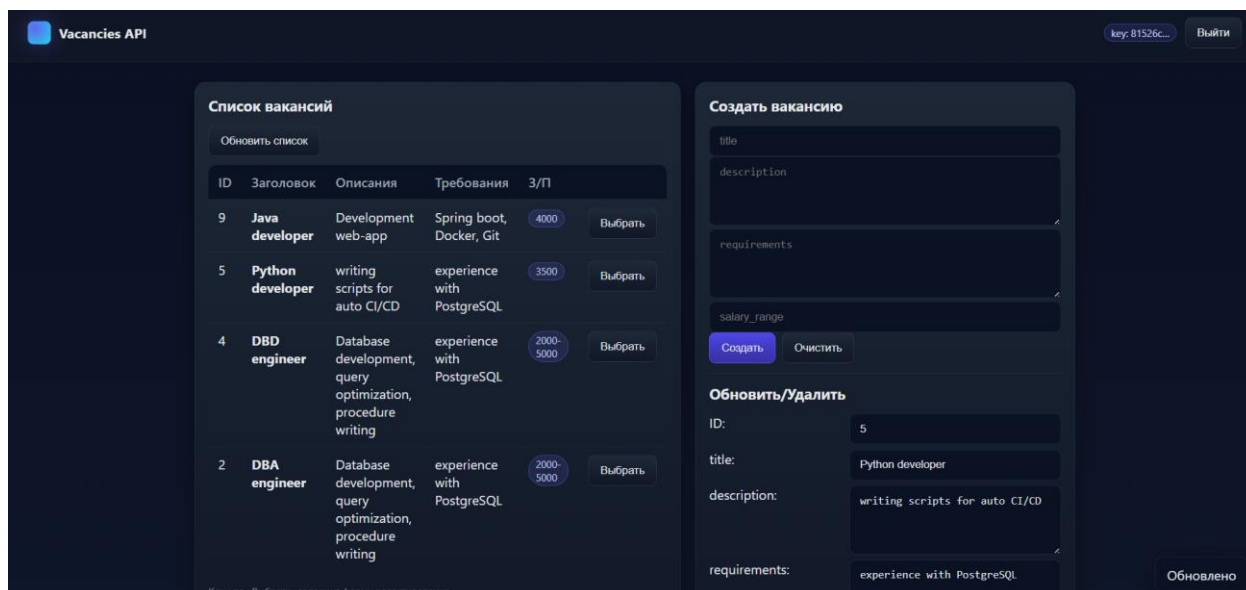


Рисунок 8 – Полное обновление вакансии (*PUT /api/vacancies/{id}*)

На рисунке 9 представлен результат выполнения операции удаления вакансии через метод *DELETE* к эндпоинту */api/vacancies/{id}*. После успешного выполнения запроса сервер возвращает статус 200 OK и подтверждающее сообщение в формате *JSON*, содержащее идентификатор удаленной записи и статус операции. Данная операция является идемпотентной - повторный запрос с тем же идентификатором возвращает аналогичный результат без ошибки.

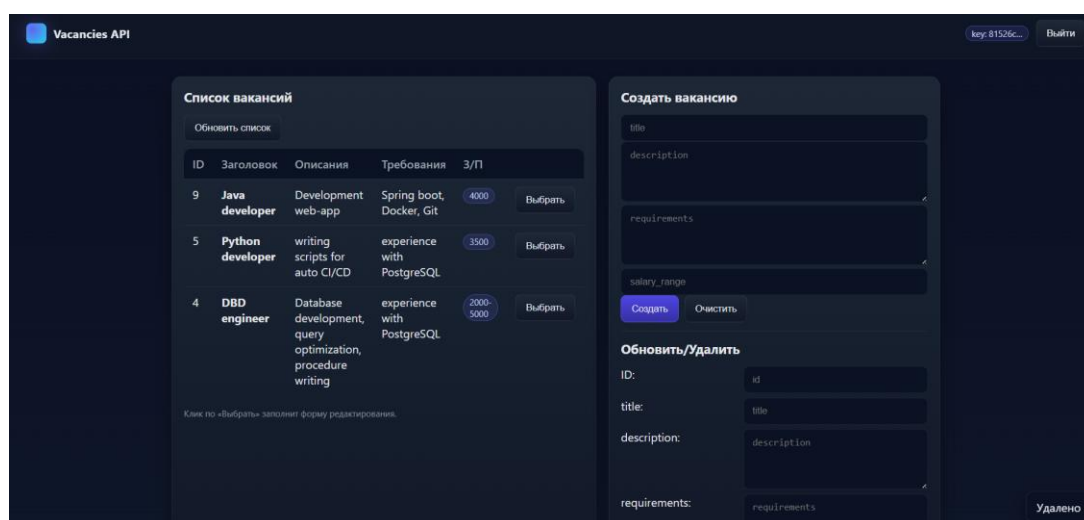


Рисунок 9 – Успешное удаление вакансии (*DELETE /api/vacancies/{id}*)

На рисунках 10 – 15 выполним тестирование в программе *Postman*.

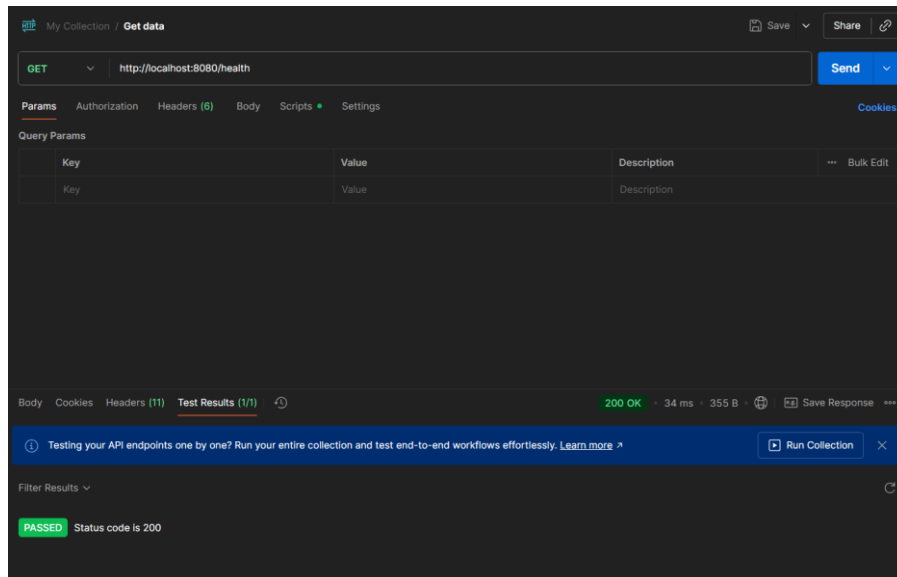


Рисунок 10 – Демонстрация здоровья сервера

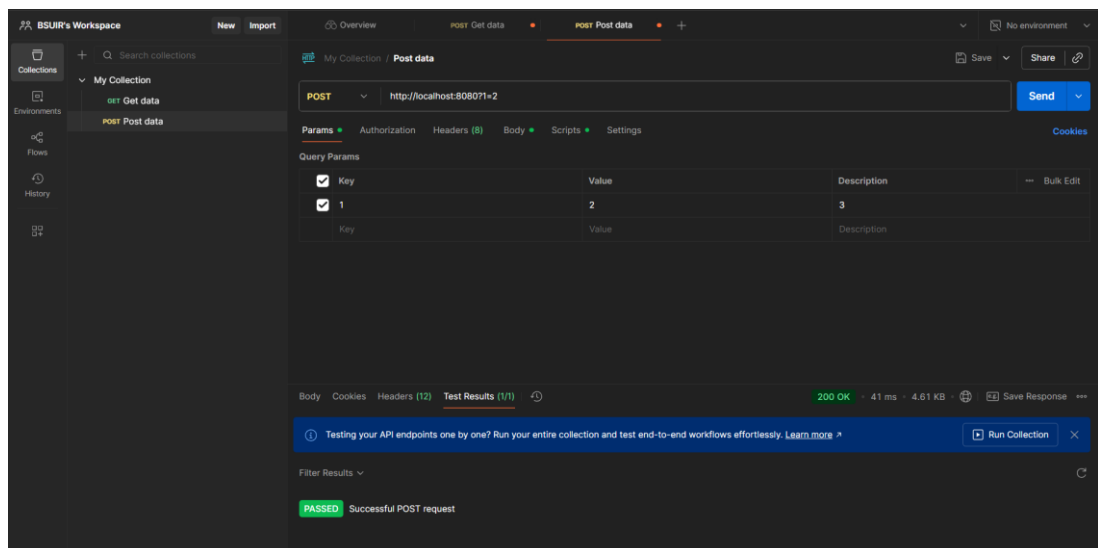


Рисунок 11 – Демонстрация *POST* запроса

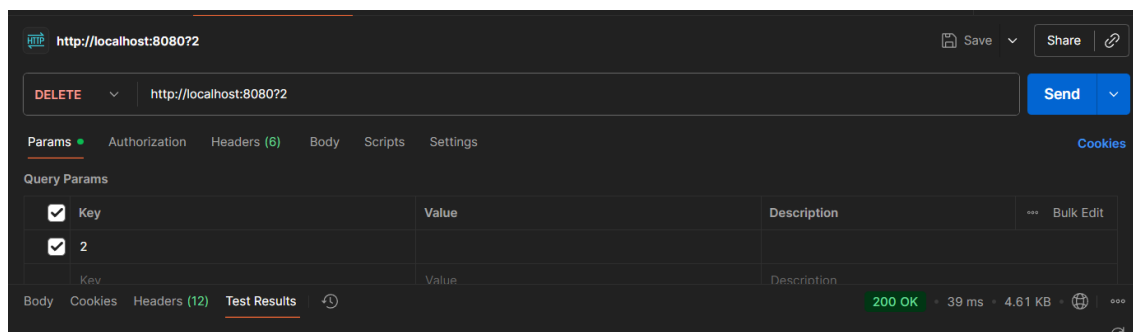


Рисунок 12 – Демонстрация *DELETE* запроса

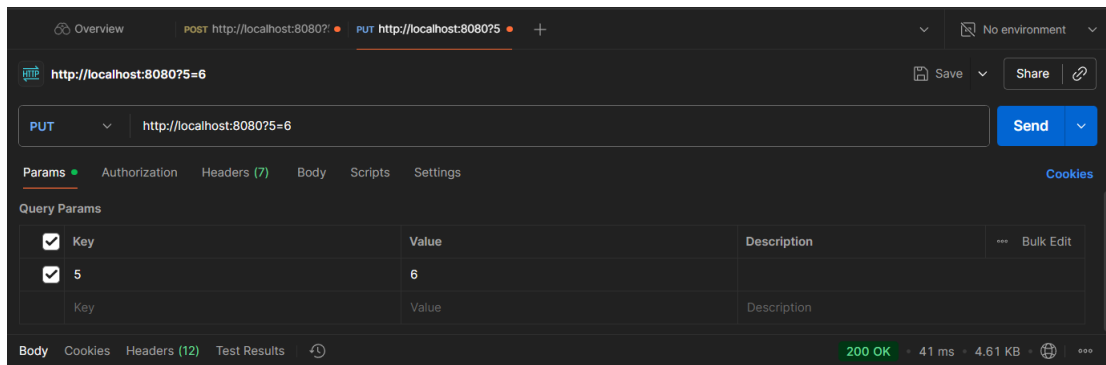


Рисунок 13 – Демонстрация запроса *PUT*

В данном примере ранее на предыдущей вкладке был выполнен запрос *POST* `key = 5, value = 5`, затем на новой вкладке выполняем замену `key = 5, value = 6`.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы был успешно спроектирован и реализован полнофункциональный *RESTful API* на языке *PHP* для управления ресурсом «Вакансии». Основной целью проекта являлось создание масштабируемого, безопасного и соответствующего принципам *REST* веб-сервиса, и все поставленные задачи были достигнуты в полном объеме. Разработанное *API* демонстрирует строгое соблюдение архитектурных ограничений *REST*, в частности принципа *Statelessness*, что обеспечивает его высокую производительность и возможность горизонтального масштабирования. Каждый компонент системы был тщательно продуман и реализован с учетом современных стандартов веб-разработки.

Основная цель достигнута: реализация полного цикла операций *CRUD*, обеспечивающего клиентам возможность создания, чтения, обновления и удаления записей о вакансиях через стандартизированный интерфейс. Архитектура приложения, построенная на паттерне *Front Controller*, доказала свою эффективность в организации централизованной обработки запросов и управления зависимостями. Особое внимание было уделено вопросам безопасности: применение подготовленных выражений *PDO* полностью исключило риск *SQL*-инъекций, а механизм аутентификации на основе хешированных *API*-ключей обеспечил надежную защиту от несанкционированного доступа.

Важным аспектом реализации стало обеспечение семантической корректности *API* через строгое соответствие *HTTP*-стандартам. Система возвращает детализированные коды состояния для различных сценариев, включая успешное выполнение операций (200, 201), клиентские ошибки (400, 401, 404) и серверные сбои (500). Унифицированный формат *JSON*-ответов и централизованная обработка ошибок значительно упрощают процесс интеграции *API* с клиентскими приложениями и последующую отладку.

Проведенное тестирование позволило познакомиться с новым инструментом для *backend*-разработки *Postman*. Оно подтвердило корректность работы всех конечных точек и надежность механизмов безопасности и валидации данных.

ПРИЛОЖЕНИЕ А

(обязательное)

Программный код приложения

С полным кодом веб-приложения можно ознакомиться по ссылке:

https://github.com/Matveiko0oOoo/itivr_lab2

Файл *index.php*:

```
<?php

declare(strict_types=1);

use App\Config\Database;
use App\Config\Auth;
use App\Models\VacancyModel;

require_once __DIR__ . '/../src/config/Database.php';
require_once __DIR__ . '/../src/config/Auth.php';
require_once __DIR__ . '/../src/models/VacancyModel.php';

header('Content-Type: application/json');

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$uri = $_SERVER['REQUEST_URI'] ?? '/';
$path = parse_url($uri, PHP_URL_PATH) ?: '/';

if ($path === '/' || $path === '/ui') {
    header('Content-Type: text/html; charset=utf-8');
    readfile(__DIR__ . '/ui.html');
    exit;
}

if (strpos($path, '/api') !== 0) {
    http_response_code(404);
    echo json_encode(['error' => 'Not found']);
    exit;
}

$pdo = App\Config\Database::getConnection();

App\Config\Auth::requireApiKey($pdo);

$segments = array_values(array_filter(explode('/', $path)));
```

```

if (count($segments) >= 2 && $segments[0] === 'api' && $segments[1] === 'vacancies') {
    $model = new App\Models\VacancyModel($pdo);

    $readJson = function (): array {
        $raw = file_get_contents('php://input');
        $data = json_decode($raw ?: '[]', true);
        if (!is_array($data)) {
            http_response_code(400);
            echo json_encode(['error' => 'Invalid JSON']);
            exit;
        }
        return $data;
    };

    if ($method === 'GET' && count($segments) === 2) {
        $rows = $model->getAll();
        echo json_encode($rows);
        exit;
    }

    if ($method === 'GET' && count($segments) === 3) {
        $id = (int)$segments[2];
        if ($id <= 0) {
            http_response_code(400);
            echo json_encode(['error' => 'Invalid id']);
            exit;
        }
        $row = $model->getById($id);
        if (!$row) {
            http_response_code(404);
            echo json_encode(['error' => 'Not found']);
            exit;
        }
        echo json_encode($row);
        exit;
    }

    if ($method === 'POST' && count($segments) === 2) {
        $data = $readJson();
        $title = isset($data['title']) && is_string($data['title']) ? trim($data['title']) : '';
        if ($title === '') {
            http_response_code(400);
            echo json_encode(['error' => 'Field title is required']);
            exit;
        }
    }
}

```

```

        $payload = [
            'title' => $title,
            'description' => isset($data['description']) ? (string)$data['description'] :
null,
            'requirements' => isset($data['requirements']) ?
(string)$data['requirements'] : null,
            'salary_range' => isset($data['salary_range']) ?
(string)$data['salary_range'] : null,
        ];
        $id = $model->create($payload);
        http_response_code(201);
        echo json_encode(['id' => $id] + $payload);
        exit;
    }

    if (($method === 'PUT' || $method === 'PATCH') && count($segments) === 3) {
        $id = (int)$segments[2];
        if ($id <= 0) {
            http_response_code(400);
            echo json_encode(['error' => 'Invalid id']);
            exit;
        }
        if (!$model->getById($id)) {
            http_response_code(404);
            echo json_encode(['error' => 'Not found']);
            exit;
        }
        $data = $readJson();
        $title = isset($data['title']) && is_string($data['title']) ? trim($data['title']) : '';
        if ($title === '') {
            http_response_code(400);
            echo json_encode(['error' => 'Field title is required']);
            exit;
        }
        $payload = [
            'title' => $title,
            'description' => isset($data['description']) ? (string)$data['description'] :
null,
            'requirements' => isset($data['requirements']) ?
(string)$data['requirements'] : null,
            'salary_range' => isset($data['salary_range']) ?
(string)$data['salary_range'] : null,
        ];
        $model->update($id, $payload);
        echo json_encode(['id' => $id] + $payload);
        exit;
    }
}

```

```

    }

    if ($method === 'DELETE' && count($segments) === 3) {
        $id = (int)$segments[2];
        if ($id <= 0) {
            http_response_code(400);
            echo json_encode(['error' => 'Invalid id']);
            exit;
        }
        if (!$model->getById($id)) {
            http_response_code(404);
            echo json_encode(['error' => 'Not found']);
            exit;
        }
        $model->delete($id);
        echo json_encode(['status' => 'deleted', 'id' => $id]);
        exit;
    }
}

```

```
http_response_code(404);
```

```
echo json_encode(['error' => 'Route not found']);
```

Файл seed_api_key.php:

```

<?php
declare(strict_types=1);

$env = getenv('APP_ENV') ?: 'production';
$dbHost = getenv('DB_HOST') ?: 'localhost';
$dbPort = getenv('DB_PORT') ?: '3306';
$dbName = getenv('DB_NAME') ?: 'vacancies_db';
$dbUser = getenv('DB_USER') ?: 'root';
$dbPass = getenv('DB_PASS') ?: '';
$outputPath = getenv('API_KEY_OUTPUT_PATH') ?: __DIR__ . '/../generated_api_key.txt';

try {
    $pdo = new
PDO("mysql:host={$dbHost};port={$dbPort};dbname={$dbName};charset=utf8mb4",
    $dbUser, $dbPass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]);
} catch (Throwable $e) {
    fwrite(STDERR, "DB connection failed: " . $e->getMessage() . "\n");
}

```

```

        exit(0);
    }

    function generateToken(int $length = 48): string {
        $bytes = random_bytes((int)ceil($length / 2));
        return substr(bin2hex($bytes), 0, $length);
    }

    $count = (int)$pdo->query('SELECT COUNT(*) FROM api_keys WHERE is_active = 1')->fetchColumn();

    $shouldGenerateNew = false;
    if ($count === 0) {
        $shouldGenerateNew = true;
    } else {
        if (!file_exists($outputPath)) {
            $shouldGenerateNew = true;
        }
    }

    if ($shouldGenerateNew) {
        $pdo->exec('UPDATE api_keys SET is_active = 0 WHERE is_active = 1');

        $plain = generateToken(48);
        $hash = password_hash($plain, PASSWORD_DEFAULT);
        $stmt = $pdo->prepare('INSERT INTO api_keys (api_key_hash, user_id, is_active)
VALUES (:hash, :user_id, 1)');
        $stmt->execute([
            ':hash' => $hash,
            ':user_id' => null,
        ]);

        file_put_contents($outputPath, "API_KEY={$plain}\n");
        chmod($outputPath, 0600);
        echo "Generated new API key written to {$outputPath}\n";
        exit(0);
    }

    if (!file_exists($outputPath)) {
        file_put_contents($outputPath, "# No plaintext available for existing keys.\n# To rotate a
new key, delete this file and restart container.\n");
        chmod($outputPath, 0600);
    }

    echo "API key already exists. Skipping seed.\n";

```

VacanciesAPI.postman_collection.json:

```
{
  "info": {
    "name": "Vacancies API",
    "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "item": [
    {
      "name": "Get all vacancies",
      "request": {
        "method": "GET",
        "header": [
          { "key": "X-API-Key", "value": "{{api_key}}" }
        ],
        "url": {
          "raw": "{{base_url}}/api/vacancies",
          "host": ["{{base_url}}"],
          "path": ["api", "vacancies"]
        }
      }
    },
    {
      "name": "Get vacancy by id",
      "request": {
        "method": "GET",
        "header": [
          { "key": "X-API-Key", "value": "{{api_key}}" }
        ],
        "url": {
          "raw": "{{base_url}}/api/vacancies/1",
          "host": ["{{base_url}}"],
          "path": ["api", "vacancies", "1"]
        }
      }
    },
    {
      "name": "Create vacancy",
      "request": {
        "method": "POST",
        "header": [
          { "key": "X-API-Key", "value": "{{api_key}}" },
          { "key": "Content-Type", "value": "application/json" }
        ],
        "body": {
          "mode": "raw",
```

```

        "raw": "{\n  \"title\": \"Backend Developer\",\n  \"description\": \"PHP, MySQL\",\n  \"requirements\": \"3+ years\",\n  \"salary_range\": \"$2000-$3000\"\n}"
      },
      "url": {
        "raw": "{{base_url}}/api/vacancies",
        "host": ["{{base_url}}"],
        "path": ["api", "vacancies"]
      }
    },
    {
      "name": "Update vacancy",
      "request": {
        "method": "PUT",
        "header": [
          { "key": "X-API-Key", "value": "{{api_key}}" },
          { "key": "Content-Type", "value": "application/json" }
        ],
        "body": {
          "mode": "raw",
          "raw": "{\n  \"title\": \"Senior Backend Developer\",\n  \"description\": \"PHP, MySQL, Docker\",\n  \"requirements\": \"5+ years\",\n  \"salary_range\": \"$3000-$4000\"\n}"
        },
        "url": {
          "raw": "{{base_url}}/api/vacancies/1",
          "host": ["{{base_url}}"],
          "path": ["api", "vacancies", "1"]
        }
      }
    },
    {
      "name": "Delete vacancy",
      "request": {
        "method": "DELETE",
        "header": [
          { "key": "X-API-Key", "value": "{{api_key}}" }
        ],
        "url": {
          "raw": "{{base_url}}/api/vacancies/1",
          "host": ["{{base_url}}"],
          "path": ["api", "vacancies", "1"]
        }
      }
    }
  ]
}

```

```

    ],
    "variable": [
        { "key": "base_url", "value": "http://localhost:8080" },
        { "key": "api_key", "value": "" }
    ]
}

```

.htaccess:

```

<IfModule mod_rewrite.c>
    RewriteEngine On

    # Allow direct access to existing files and directories
    RewriteCond %{REQUEST_FILENAME} -f [OR]
    RewriteCond %{REQUEST_FILENAME} -d
    RewriteRule ^ - [L]

    # Route everything else to index.php
    RewriteRule ^ index.php [QSA,L]
</IfModule>

```

```

# Security headers
<IfModule mod_headers.c>
    Header set X-Content-Type-Options "nosniff"
    Header set X-Frame-Options "SAMEORIGIN"
    Header set X-XSS-Protection "1; mode=block"
</IfModule>

```

ui.html:

```

<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vacancies API Demo</title>
    <style>
        :root {
            --bg: #0f172a;
            --surface: #111827;
            --card: #1f2937;
            --muted: #9ca3af;
            --text: #e5e7eb;
            --accent: #6366f1;
            --accent-2: #22d3ee;
            --danger: #ef4444;

```

```

        --ok: #22c55e;
    }
    *{ box-sizing: border-box; }
    body { margin: 0; font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto,
    Ubuntu, Cantarell, Noto Sans, Helvetica Neue, Arial, "Apple Color Emoji", "Segoe UI Emoji";
    background: linear-gradient(180deg, #0b1024, #0f172a); color: var(--text); }
    .header { display: flex; align-items: center; justify-content: space-between; padding:
    16px 24px; border-bottom: 1px solid rgba(255,255,255,0.06); background: rgba(17,24,39,0.7);
    backdrop-filter: blur(6px); position: sticky; top: 0; z-index: 10; }
    .brand { display: flex; align-items: center; gap: 10px; font-weight: 700; letter-
    spacing: 0.3px; }
    .brand .logo { width: 28px; height: 28px; border-radius: 7px; background: radial-
    gradient(circle at 30% 30%, var(--accent), var(--accent-2)); box-shadow: 0 0 24px
    rgba(99,102,241,0.4); }
    .container { max-width: 1100px; margin: 24px auto; padding: 0 16px; }
    .toolbar { display: flex; gap: 10px; align-items: center; }
    .btn { border: 1px solid rgba(255,255,255,0.08); background: linear-
    gradient(180deg, #1f2937, #111827); color: var(--text); padding: 10px 14px; border-radius: 8px;
    cursor: pointer; transition: .15s ease; }
    .btn:hover { transform: translateY(-1px); box-shadow: 0 6px 16px rgba(0,0,0,.35);
    }

    .btn.primary { background: linear-gradient(180deg, #4f46e5, #3730a3); border-
    color: rgba(99,102,241,0.5); }
    .btn.danger { background: linear-gradient(180deg, #b91c1c, #7f1d1d); border-
    color: rgba(239,68,68,0.5); }
    .input, .textarea { width: 100%; background: #0b1224; color: var(--text); border:
    1px solid rgba(255,255,255,0.08); border-radius: 8px; padding: 10px 12px; outline: none; }
    .textarea { min-height: 84px; resize: vertical; }
    .grid { display: grid; grid-template-columns: 1.1fr 1fr; gap: 18px; }
    @media (max-width: 900px) { .grid { grid-template-columns: 1fr; } }
    .card { background: linear-gradient(180deg, rgba(31,41,55,0.9),
    rgba(17,24,39,0.9)); border: 1px solid rgba(255,255,255,0.06); border-radius: 14px; padding:
    16px; box-shadow: 0 10px 30px rgba(0,0,0,.35); }
    .card h3 { margin: 0 0 12px 0; font-size: 18px; }
    .table { width: 100%; border-collapse: collapse; overflow: hidden; border-radius:
    10px; }
    .table thead th { text-align: left; font-weight: 600; color: var(--muted); padding:
    10px 12px; background: #0b1224; }
    .table tbody tr { border-top: 1px solid rgba(255,255,255,0.06); }
    .table td { padding: 10px 12px; vertical-align: top; }
    .badge { display: inline-block; padding: 2px 8px; border-radius: 999px;
    background: rgba(99,102,241,0.15); border: 1px solid rgba(99,102,241,0.4); color: #c7d2fe;
    font-size: 12px; }
    .kv { display: grid; grid-template-columns: 160px 1fr; gap: 6px 10px; }
    .footer { color: var(--muted); font-size: 12px; margin-top: 18px; opacity: .8; }
    .sep { height: 1px; background: rgba(255,255,255,0.06); margin: 12px 0; }

```

```

        #login { position: fixed; inset: 0; display: flex; align-items: center; justify-content:
        center; background: radial-gradient(ellipse at top, rgba(79,70,229,0.15), transparent 40%),
        radial-gradient(ellipse at bottom, rgba(34,211,238,0.1), transparent 40%), rgba(2,6,23,0.85);
        backdrop-filter: blur(6px); }
        .login-box { width: 420px; max-width: 92vw; background: linear-gradient(180deg,
        rgba(17,24,39,0.95), rgba(2,6,23,0.95)); border: 1px solid rgba(255,255,255,0.08); border-
        radius: 16px; padding: 20px; box-shadow: 0 20px 60px rgba(0,0,0,.55); }
        .login-box h2 { margin: 0 0 10px 0; }
        .hint { color: var(--muted); font-size: 13px; }
        .error { color: #fecaca; background: rgba(239,68,68,0.12); border: 1px solid
        rgba(239,68,68,0.35); padding: 8px 10px; border-radius: 8px; margin-top: 8px; display:none; }

        .toast { position: fixed; right: 16px; bottom: 16px; background: #0b1224; color:
        var(--text); border: 1px solid rgba(255,255,255,0.08); padding: 10px 14px; border-radius: 10px;
        display: none; box-shadow: 0 10px 30px rgba(0,0,0,.4); }
    </style>
</head>
<body>
    <header class="header">
        <div class="brand"><div class="logo"></div> Vacancies API</div>
        <div class="toolbar">
            <span id="who" class="badge" title="Текущий ключ">—</span>
            <button class="btn" id="btnLogout" onclick="logout()"
            style="display:none">Выйти</button>
        </div>
    </header>

    <div id="login">
        <div class="login-box">
            <h2>Вход</h2>
            <p class="hint">Введите ваш API-ключ (заголовок <code>X-API-
            Key</code>).</p>
            <input id="loginKey" class="input" placeholder="API_KEY..." />
            <div class="error" id="loginError">Неверный ключ. Попробуйте
            снова.</div>
            <div style="display:flex; gap:10px; margin-top: 10px;">
                <button class="btn primary"
                onclick="doLogin()">Войти</button>
                <button class="btn" onclick="prefillFromFile()">Подставить из
                файла</button>
            </div>
            <div class="footer">Подсказка: ключ сохраняется локально в браузере
            и используется для всех запросов.</div>
        </div>
    </div>

```

```

<div class="container" id="app" style="display:none">
  <div class="grid">
    <div class="card">
      <h3>Список вакансий</h3>
      <div style="display:flex; gap:8px; margin-bottom:10px;">
        <button class="btn" onclick="loadAll()">Обновить
список</button>
      </div>
      <div style="overflow:auto">
        <table class="table" id="tbl">
          <thead>
            <tr>
              <th>ID</th>
              <th>Заголовок</th>
              <th>Описания</th>
              <th>Требования</th>
              <th>З/П</th>
              <th></th>
            </tr>
          </thead>
          <tbody id="tbody"></tbody>
        </table>
      </div>
      <div class="footer">Клик по «Выбрать» заполнит форму
редактирования.</div>
    </div>
    <div class="card">
      <h3>Создать вакансию</h3>
      <input id="title" class="input" placeholder="title" />
      <textarea id="description" class="textarea"
placeholder="description"></textarea>
      <textarea id="requirements" class="textarea"
placeholder="requirements"></textarea>
      <input id="salary_range" class="input"
placeholder="salary_range" />
      <div style="display:flex; gap:8px;">
        <button class="btn primary"
onclick="createVacancy()">Создать</button>
        <button class="btn"
onclick="clearCreate()">Очистить</button>
      </div>
      <div class="sep"></div>
      <h3>Обновить/Удалить</h3>
      <div class="kv">
        <div>ID:</div>

```

```

        <input id="upd_id" class="input" placeholder="id" />
        <div>title:</div>
        <input id="upd_title" class="input" placeholder="title" />
        <div>description:</div>
        <textarea      id="upd_description"      class="textarea"
placeholder="description"></textarea>
        <div>requirements:</div>
        <textarea      id="upd_requirements"      class="textarea"
placeholder="requirements"></textarea>
        <div>salary_range:</div>
        <input          id="upd_salary_range"          class="input"
placeholder="salary_range" />
    </div>
    <div style="display:flex; gap:8px; margin-top:8px;">
        <button          class="btn          primary"
onclick="updateVacancy()">Обновить</button>
        <button          class="btn          danger"
onclick="deleteVacancy()">Удалить</button>
        <button          class="btn"
onclick="clearUpdate()">Очистить</button>
    </div>
</div>
</div>
<div id="toast" class="toast"></div>

<script>
    const KEY_NAME = 'vacancies_api_key';

    function showToast(text) {
        const t = document.getElementById('toast');
        t.textContent = text;
        t.style.display = 'block';
        setTimeout(() => t.style.display = 'none', 2600);
    }

    function setLoggedIn(key) {
        localStorage.setItem(KEY_NAME, key);
        document.getElementById('who').textContent = `key: ${key.slice(0,6)}...`;
        document.getElementById('btnLogout').style.display = 'inline-block';
        document.getElementById('login').style.display = 'none';
        document.getElementById('app').style.display = 'block';
        loadAll();
    }

```

```

function logout() {
    localStorage.removeItem(KEY_NAME);
    document.getElementById('btnLogout').style.display = 'none';
    document.getElementById('login').style.display = 'flex';
    document.getElementById('app').style.display = 'none';
    showToast('Вы вышли');
}

function currentKey() {
    return localStorage.getItem(KEY_NAME) || '';
}

function getHeaders() {
    const key = currentKey();
    return {
        'Content-Type': 'application/json',
        'X-API-Key': key
    };
}

async function api(path, options = {}) {
    const res = await fetch(path, { ...options, headers: { ...(options.headers||{}),
...getHeaders() } });
    if (res.status === 401) {
        showToast('Не авторизовано. Войдите заново');
        logout();
        throw new Error('Unauthorized');
    }
    let data = null;
    try { data = await res.json(); } catch(e) { /* ignore */ }
    if (!res.ok) {
        throw new Error(data && data.error ? data.error : `HTTP
${res.status}`);
    }
    return data;
}

async function prefillFromFile() {
    try {
        const txt = await fetch('/.generated_api_key.txt').then(r => r.text());
        const m = txt.match(/API_KEY=(.+)/);
        if (m) {
            document.getElementById('loginKey').value = m[1].trim();
            showToast('Ключ подставлен из файла');
            return;
        }
    }
}

```

```

        showToast('Файл не найден или пуст');
    } catch (e) {
        showToast('Не удалось прочитать файл ключа');
    }
}

async function doLogin() {
    const key = document.getElementById('loginKey').value.trim();
    const err = document.getElementById('loginError');
    err.style.display = 'none';
    if (!key) { err.style.display = 'block'; return; }
    try {
        await fetch('/api/vacancies', { headers: { 'X-API-Key': key } }).then(r
=> { if (r.status === 401) throw new Error('401'); return r; });
        setLoggedIn(key);
        showToast('Вход выполнен');
    } catch(e) {
        err.style.display = 'block';
    }
}

function setUpdateForm(v) {
    document.getElementById('upd_id').value = v.id;
    document.getElementById('upd_title').value = v.title || '';
    document.getElementById('upd_description').value = v.description || '';
    document.getElementById('upd_requirements').value = v.requirements || '';
    document.getElementById('upd_salary_range').value = v.salary_range ||
";
}

function clearCreate(){
    document.getElementById('title').value="";
    document.getElementById('description').value="";
    document.getElementById('requirements').value="";
    document.getElementById('salary_range').value="";
}

function clearUpdate(){
    document.getElementById('upd_id').value="";
    document.getElementById('upd_title').value="";
    document.getElementById('upd_description').value="";
    document.getElementById('upd_requirements').value="";
    document.getElementById('upd_salary_range').value="";
}

async function loadAll(){
    try {

```

```

const data = await api('/api/vacancies');
const tbody = document.getElementById('tbody');
tbody.innerHTML = '';
for (const v of data) {
    const tr = document.createElement('tr');
    tr.innerHTML = `
        <td>${v.id}</td>

        <td><strong>${escapeHtml(v.title||'')}</strong> </td>
        <td>${escapeHtml(v.description||'')}</td>
        <td>${escapeHtml(v.requirements||'')}</td>
        <td><span
class="badge">${escapeHtml(v.salary_range||'')}</span> </td>
        <td><button                                class="btn"
onclick='selectRow(${JSON.stringify(v).replace(/'/g, "&#39;")})'>Выбрать</button> </td>
    `;
    tbody.appendChild(tr);
}
} catch(e) {
    showToast(e.message);
}
}

function selectRow(v) { setUpdateForm(v); showToast('Заполнено для
редактирования'); }

async function createVacancy(){
    const body = {
        title: document.getElementById('title').value,
        description: document.getElementById('description').value,
        requirements: document.getElementById('requirements').value,
        salary_range: document.getElementById('salary_range').value,
    };
    try {
        const data = await api('/api/vacancies', { method: 'POST', body:
JSON.stringify(body) });
        showToast('Создано: id ' + data.id);
        clearCreate();
        loadAll();
    } catch(e) { showToast(e.message); }
}

async function updateVacancy(){
    const id = document.getElementById('upd_id').value;
    if (!id) { showToast('Укажите id'); return; }
    const body = {

```

```

        title: document.getElementById('upd_title').value,
        description: document.getElementById('upd_description').value,
        requirements:
document.getElementById('upd_requirements').value,
        salary_range:
document.getElementById('upd_salary_range').value,
    };
    try {
        await api('/api/vacancies/' + encodeURIComponent(id), { method:
'PUT', body: JSON.stringify(body) });
        showToast('Обновлено');
        loadAll();
    } catch(e) { showToast(e.message); }
}

async function deleteVacancy(){
    const id = document.getElementById('upd_id').value;
    if (!id) { showToast('Укажите id'); return; }
    if (!confirm('Удалить вакансию #' + id + '?')) return;
    try {
        await api('/api/vacancies/' + encodeURIComponent(id), { method:
'DELETE' });

        showToast('Удалено');
        clearUpdate();
        loadAll();
    } catch(e) { showToast(e.message); }
}

function escapeHtml(s){
    return (s||'').replace(/[\&<>"]/g,
    ({'&':'&','<':'<','>':'>','"':'"','\':'&#39;'}[c]));
}

(function init(){
    const key = currentKey();
    if (key) { setLoggedIn(key); }
})();
</script>
</body>
</html>

```

init.sql:

```

CREATE TABLE IF NOT EXISTS api_keys (
    id INT AUTO_INCREMENT PRIMARY KEY,
    api_key_hash VARCHAR(255) NOT NULL,

```

```

user_id INT NULL,
is_active TINYINT(1) NOT NULL DEFAULT 1,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

```

CREATE TABLE IF NOT EXISTS vacancies (
id INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(255) NOT NULL,
description TEXT NULL,
requirements TEXT NULL,
salary_range VARCHAR(100) NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NULL DEFAULT NULL ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

Auth.php:

```

<?php

declare(strict_types=1);

namespace App\Config;

use PDO;

class Auth
{
    public static function requireApiKey(PDO $pdo): void
    {
        $headers = function_exists('getallheaders') ? getallheaders() : [];
        $apiKey = $headers['X-API-Key'] ?? $headers['x-api-key'] ?? null;
        if (!$apiKey) {
            self::unauthorized('Missing API key');
        }

        $stmt = $pdo->prepare('SELECT api_key_hash, is_active FROM api_keys
WHERE is_active = 1');
        $stmt->execute();
        $keys = $stmt->fetchAll();
        $ok = false;
        foreach ($keys as $row) {
            if (password_verify($apiKey, $row['api_key_hash'])) {
                $ok = true; break;
            }
        }
        if (!$ok) {

```

```

        self::unauthorized('Invalid API key');
    }
}

private static function unauthorized(string $message): void
{
    http_response_code(401);
    header('Content-Type: application/json');
    echo json_encode(['error' => $message]);
    exit;
}
}

```

Database.php:

```

<?php

declare(strict_types=1);

namespace App\Config;

use PDO;
use PDOException;

class Database
{
    public static function getConnection(): PDO
    {
        $host = getenv('DB_HOST') ?: 'localhost';
        $port = getenv('DB_PORT') ?: '3306';
        $name = getenv('DB_NAME') ?: 'vacancies_db';
        $user = getenv('DB_USER') ?: 'root';
        $pass = getenv('DB_PASS') ?: '';

        $dsn = "mysql:host={$host};port={$port};dbname={$name};charset=utf8mb4";
        try {
            $pdo = new PDO($dsn, $user, $pass, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE =>
PDO::FETCH_ASSOC,
            ]);
            return $pdo;
        } catch (PDOException $e) {
            http_response_code(500);
            header('Content-Type: application/json');
            echo json_encode(['error' => 'Database connection failed']);
        }
    }
}

```

```

        exit;
    }
}
}

```

VacancyModel.php:

```

<?php

declare(strict_types=1);

namespace App\Models;

use PDO;

class VacancyModel
{
    private PDO $pdo;

    public function __construct(PDO $pdo)
    {
        $this->pdo = $pdo;
    }

    public function getAll(): array
    {
        $stmt = $this->pdo->query('SELECT id, title, description, requirements,
salary_range, created_at, updated_at FROM vacancies ORDER BY id DESC');
        return $stmt->fetchAll();
    }

    public function getId(int $id): ?array
    {
        $stmt = $this->pdo->prepare('SELECT id, title, description, requirements,
salary_range, created_at, updated_at FROM vacancies WHERE id = :id');
        $stmt->execute([':id' => $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    public function create(array $data): int
    {
        $stmt = $this->pdo->prepare('INSERT INTO vacancies (title, description,
requirements, salary_range) VALUES (:title, :description, :requirements, :salary_range)');
        $stmt->execute([
            ':title' => $data['title'],

```

```

        ':description' => $data['description'] ?? null,
        ':requirements' => $data['requirements'] ?? null,
        ':salary_range' => $data['salary_range'] ?? null,
    ]);
    return (int)$this->pdo->lastInsertId();
}

public function update(int $id, array $data): bool
{
    $stmt = $this->pdo->prepare('UPDATE vacancies SET title = :title, description =
:description, requirements = :requirements, salary_range = :salary_range WHERE id = :id');
    return $stmt->execute([
        ':title' => $data['title'],
        ':description' => $data['description'] ?? null,
        ':requirements' => $data['requirements'] ?? null,
        ':salary_range' => $data['salary_range'] ?? null,
        ':id' => $id,
    ]);
}

public function delete(int $id): bool
{
    $stmt = $this->pdo->prepare('DELETE FROM vacancies WHERE id = :id');
    return $stmt->execute([':id' => $id]);
}
}

```

docker-compose.yml:

```

services:
  app:
    build: .
    container_name: php_api_app
    ports:
      - "8080:80"
    environment:
      - APP_ENV=local
      - DB_HOST=db
      - DB_PORT=3306
      - DB_NAME=vacancies_db
      - DB_USER=vacancies_user
      - DB_PASS=vacancies_pass
      - API_KEY_OUTPUT_PATH=/var/www/html/.generated_api_key.txt
    depends_on:
      - db
    volumes:

```

```

- ./:/var/www/html
db:
image: mysql:8.0
container_name: php_api_db
environment:
- MYSQL_DATABASE=vacancies_db
- MYSQL_USER=vacancies_user
- MYSQL_PASSWORD=vacancies_pass
- MYSQL_ROOT_PASSWORD=rootpass
ports:
- "33060:3306"
volumes:
- db_data:/var/lib/mysql
- ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
volumes:
db_data:

```

Dockerfile:

```
FROM php:8.2-apache
```

```
RUN docker-php-ext-install pdo pdo_mysql
```

```
RUN a2enmod rewrite headers
```

```
ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
```

```
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/000-default.conf \
```

```
&& sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/apache2.conf \
```

```
&& sed -ri -e 's/AllowOverride None/AllowOverride All/g' /etc/apache2/apache2.conf \
```

```
&& echo "ServerName localhost" > /etc/apache2/conf-available/servername.conf \
```

```
&& a2enconf servername
```

```
COPY ./var/www/html/
```

```
RUN chown -R www-data:www-data /var/www/html \
```

```
&& find /var/www/html -type d -exec chmod 755 {} \;
```

```
&& find /var/www/html -type f -exec chmod 644 {} \;
```

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 CMD curl -fsS http://localhost/health || exit 1
```

```
COPY docker-entrypoint.sh /usr/local/bin/docker-entrypoint.sh
```

```
RUN chmod +x /usr/local/bin/docker-entrypoint.sh
```

```
EXPOSE 80
```

```
ENTRYPOINT ["/usr/local/bin/docker-entrypoint.sh"]
```